

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle TI

PHP avancé : structures de contrôle et interfaçage avec MySQL

Module 1 : Programmation

Cette fiche propose des problèmes de synthèse mobilisant l'ensemble des notions du chapitre : algorithmique PHP, gestion rigoureuse des erreurs MySQLi et traitement sécurisé des données.

EXERCICE 1 Architecture de connexion et gestion d'erreurs robuste

On souhaite concevoir un module de connexion réutilisable `db_connect.php` pour une application de gestion de stock. La base de données `stock_db` contient une table `produits(id_prod, nom, prix, qte_stock, date_ajout)`.

1. Écrire le code complet de `db_connect.php` utilisant `mysqli_connect`. Le script doit : sélectionner la base `stock_db`; configurer le jeu de caractères `utf8mb4`; en cas d'échec (connexion ou sélection), journaliser l'erreur dans `logs/db_errors.log` (format : `[YYYY-MM-DD HH:MM:SS] Erreur : ...`) puis arrêter l'exécution via `die()` avec un message utilisateur générique. Ne pas afficher l'erreur technique à l'écran.
2. Expliquer pourquoi l'utilisation de `die()` dans un contexte de production professionnelle est souvent critiquée. Proposer une alternative structurelle (sans code) permettant de séparer la gestion de l'erreur de l'arrêt du script.
3. Dans un script principal `index.php`, on inclut ce module. Écrire la structure de contrôle (avec `require_once`) qui vérifie si la variable de connexion `$link` est une ressource `mysqli` valide avant d'exécuter la logique métier. Justifier le choix de `require_once` over `include`.

EXERCICE 2 Traitement transactionnel simulé et structures de contrôle

La table `commandes(id_cmd, id_prod, qte, statut, date_cmd)` gère les commandes. Un script `traiter_panier.php` reçoit un tableau associatif `$panier = ['id_prod' => qte, ...]` en POST (JSON décodé). Le traitement doit être atomique : soit toutes les lignes sont insérées et le stock décrémenté, soit rien n'est fait.

1. Écrire l'algorithme (pseudo-code ou PHP commenté) utilisant une boucle `foreach` pour valider le panier avant toute écriture SQL : vérifier que `qte > 0`, que `id_prod` existe en base (requête `SELECT qte_stock`) et que le stock est suffisant. Stocker les erreurs dans un tableau `$erreurs`.
2. Si `$erreurs` est vide, écrire la séquence d'instructions `mysqli_query` pour : 1. Désactiver l'auto-commit (`SET autocommit=0`). 2. Boucler sur le panier pour `INSERT INTO commandes` et `UPDATE produits SET qte_stock = qte_stock - ?`. 3. Valider (`COMMIT`) ou

Annuler (ROLLBACK) selon le succès de toutes les requêtes. 4. Réactiver l'auto-commit. Gérer les erreurs de chaque `mysqli_query` sans `die()`.

3. Analyser la faille de sécurité majeure si les valeurs sont concaténées directement dans la chaîne SQL (ex : "INSERT ... VALUES (*id*,*qte*)"). Sans utiliser les requêtes préparées (hors programme), décrire précisément la fonction native `mysqli` à utiliser pour sécuriser les variables scalaires avant concaténation et donner son prototype.

EXERCICE 3 Génération de rapport dynamique et formatage conditionnel

On doit afficher un tableau HTML récapitulatif des ventes du mois courant. La requête `SELECT p.nom, c.qte, c.date_cmd, p.prix FROM commandes c JOIN produits p ON c.id_prod=p.id_prod WHERE MONTH(c.date_cmd)=MONTH(NOW())` retourne un jeu de résultats `$result`.

1. Écrire la boucle `while` avec `mysqli_fetch_assoc` pour parcourir `$result`. À l'intérieur, calculer le total ligne (`qte * prix`) et accumuler le total général. Utiliser `date('d/m/Y', strtotime($row['date_cmd']))` pour formater la date.
2. Dans le tableau HTML, la colonne Statut Stock doit afficher : "Rupture" si `qte_stock == 0` (nécessite une sous-requête ou jointure adaptée), "Critique (<5)" si `qte_stock < 5`, "OK" sinon. Implémenter cette logique avec une structure `switch` sur une expression booléenne (`switch(true)`).
3. Après le tableau, afficher le total général formaté avec 2 décimales et le symbole FCFA. Si `mysqli_num_rows($result) == 0`, afficher un message "Aucune vente ce mois-ci" à la place du tableau (structure `if/else` englobant).
4. Fermer proprement les ressources. Discuter de l'impact performance de l'exécution d'une sous-requête `SELECT qte_stock` à l'intérieur de la boucle `while` (N+1 problème) versus une jointure unique (`LEFT JOIN`) dans la requête initiale.

EXERCICE 4 Analyse critique et refactorisation d'un script legacy

On récupère le script suivant `supprimer_produit.php` :

```
<?php
$id = $_GET['id'];
$link = mysqli_connect("localhost", "root", "", "stock_db");
$sql = "DELETE FROM produits WHERE id_prod = $id";
mysqli_query($link, $sql);
echo "Produit $id supprimé.";
mysqli_close($link);
?>
```

1. Identifier et lister cinq vulnérabilités ou mauvaises pratiques majeures (sécurité, robustesse, UX, architecture) présentes dans ce code, en les reliant aux fonctions du cours (`die`, `mysqli_query`, structures de contrôle, `_GET`).
2. Réécrire complètement ce script en appliquant les corrections : vérification de l'existence de `id` (entier positif), connexion sécurisée (fichier externe), protection contre l'injection SQL (fonction citée ex. 2.3), exécution conditionnelle, message de retour différencié (succès / introuvable / erreur SQL) via `mysqli_affected_rows`, redirection header vers `liste.php?status=...` au lieu d'`echo`.
3. Question ouverte : Dans une architecture MVC moderne, ce script correspondrait à quel composant ? Justifier pourquoi la logique SQL ne devrait pas s'y trouver et vers quelle couche elle devrait être déplacée.